

Wipro Capstone Project 3

System Monitor Tool –

SOUMANSU SEKHAR NAYAK (2141014141)

Objective: Create a system monitor tool in C++ that displays real-time information about system processes, memory usage, and CPU load, similar to the 'top' command.

Day-wise Tasks:

Day 1: Design UI layout and gather system data using system calls.

```
#include <iostream> using
```

```
namespace std; #include
```

```
<sys/sysinfo.h> void
```

```
displayMemoryInfo(){
```

```
struct sysinfo info; if
```

```
(sysinfo(&info)==0){
```

```
    cout<< "Total used RAM (info.totalram in MB):
```

```
"<<info.totalram/(1024*1024)<<"MB\n";
```

```
    cout<< "Total used RAM (info.totalram in GB):
```

```
"<<(info.totalram/(1024*1024))/1024<<"GB\n";
```

```
    cout<< "Total unused RAM (info.freeram in MB):
```

```
"<<info.freeram/(1024*1024)<<"MB\n";
```

```
    cout<< "Total shared RAM (info.sharedram in MB):
```

```
"<<info.sharedram/(1024*1024)<<"MB\n";
```

```
    }
```

```
}
```

```
int main(){
```

```
    displayMemoryInfo();
```

```
    return 0;
```

```
}
```

Day 1 Output

```
student@D001-38:~/Documents/capstone-3$ g++ --std=c++17 task1.cpp -o task1
student@D001-38:~/Documents/capstone-3$ ./task1
Total used RAM (info.totalram in MB): 7681MB
Total used RAM (info.totalram in GB): 7GB
Total unused RAM (info.freeram in MB): 3839MB
Total shared RAM (info.sharedram in MB): 303MB
student@D001-38:~/Documents/capstone-3$
```

Day 2: Display process list with CPU and memory usage.

Process List

```
#include <iostream>
#include <filesystem>
#include <fstream>
#include <sstream> #include <algorithm> using namespace
std; namespace fs = std::filesystem; bool isNumber(const
string &s){    return  !s.empty() && all_of(s.begin(), s.end(),
::isdigit);
}

string getProcessName(int pid){    ifstream
file("/proc/" + to_string(pid) + "/stat");    string line,
processName;    if(file.is_open()){
getline(file,line);    istringstream ss(line);

        string token;

        int count=0;
while(ss >> token){

            count++;

            if(count==2){

processName=token;
```

```

        break;
    }
}
}
return processName;
}

int main(){
    cout<<"Active processes: "<<endl;    for (const
    auto &entry : fs::directory_iterator("/proc")){
        if(entry.is_directory()){            string
        filename=entry.path().filename().string();            if
        (isNumber(filename)){                int pid=stoi(filename);
            string processName = getProcessName(pid);
            cout<<"PID: "<<pid<<"|Name: " << processName<<endl;
                }
        }
    }
    return 0;
}

```

CPU and Memory Usage

```

#include <iostream>
#include <fstream>
#include <sstream>
#include <thread> #include <chrono> using namespace std; struct CPUData{
    long user, nice, system, idle, iowait, irq, softirq, steal, guest, guest_nice;
};

```

```

CPUData getCPUData(){
ifstream file("/proc/stat");  string
line;

    CPUData cpu={};

    if (file.is_open()){
getline(file,line);          istringstream
ss(line);                    string cpuLabel;

        ss >> cpuLabel >> cpu.user >> cpu.nice >> cpu.system >> cpu.idle >>
cpu.iowait >> cpu irq >> cpu.softirq >> cpu.steal >> cpu.guest >> cpu.guest_nice;

    }

    return cpu;
}

double calculateCPUUsage (CPUData prev, CPUData current){
long prevIdle= prev.idle + prev.iowait;    long currIdle=
current.idle + current.iowait;

    long prevTotal = prev.user + prev.nice + prev.system + prev.idle + prev.iowait +
prev.irq + prev.softirq + prev.steal;
    long currTotal = current.user + current.nice + current.system + current.idle +
current.iowait + current.irq + current.softirq + current.steal;    long totalDiff =
currTotal - prevTotal;    long idleDiff = currIdle - prevIdle; return ((totalDiff -
idleDiff) * 100.0) / totalDiff;
}

int main(){

    CPUData cpu= getCPUData();    cout << "User: " <<
cpu.user << endl;    cout << "Nice: " << cpu.nice << endl;
cout << "System: " << cpu.system << endl;    cout << "Idle: " <<
cpu.idle << endl;    cout << "IOwait: " << cpu.iowait << endl;
cout << "IRQ: " << cpu.irq << endl;    cout << "SoftIRQ: " <<
cpu.softirq << endl;    cout << "Steal: " << cpu.steal << endl;
cout << "Guest: " << cpu.guest << endl;    cout << "Guest Nice:

```

```

" << cpu.guest_nice << endl;    CPUData prevData =
getCPUData();    this_thread::sleep_for(chrono::seconds(1));
CPUData currData = getCPUData();    double cpuUsage=
calculateCPUUsage (prevData, currData);    cout << "CPU
Usage: "<<cpuUsage<<endl;

    return 0;
}

```

Day 2 Output (Process List)

```

student@D001-38:~/Documents/capstone-3$ g++ --std=c++17 task2_2.cpp -o task2_2
student@D001-38:~/Documents/capstone-3$ ./task2_2
Active processes:
PID: 1|Name: (systemd)
PID: 2|Name: (kthreadd)
PID: 3|Name: (rcu_gp)
PID: 4|Name: (rcu_par_gp)
PID: 5|Name: (slub_flushwq)
PID: 6|Name: (netns)
PID: 8|Name: (kworker/0:0H-events_highpri)
PID: 10|Name: (mm_percpu_wq)
PID: 11|Name: (rcu_tasks_rude_)
PID: 12|Name: (rcu_tasks_trace)
PID: 13|Name: (ksoftirqd/0)
PID: 14|Name: (rcu_sched)
PID: 15|Name: (migration/0)
PID: 16|Name: (idle_inject/0)
PID: 18|Name: (cpuhp/0)
PID: 19|Name: (cpuhp/1)
PID: 20|Name: (idle_inject/1)
PID: 21|Name: (migration/1)
PID: 22|Name: (ksoftirqd/1)
PID: 23|Name: (kworker/1:0-cgroup_destroy)

```

Day 2 Output (CPU and Memory Usage)

```

student@D001-38:~/Documents/capstone-3$ g++ --std=c++17 task2.cpp -o task2
student@D001-38:~/Documents/capstone-3$ ./task2
User: 155898
Nice: 574
System: 34903
Idle: 21088283
IOwait: 11435
IRQ: 0
SoftIRQ: 835
Steal: 0
Guest: 0
Guest Nice: 0
CPU Usage: 0.333333
student@D001-38:~/Documents/capstone-3$ █

```

Day 3: Implement process sorting by CPU and memory usage.

```
#include <iostream>
#include <filesystem>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <vector>
#include <dirent.h>
#include <unistd.h> #include
<csignal> using namespace
std; namespace
fs=std::filesystem; struct
ProcessInfo{
    int pid;      string
name;      double
cpuUsage;  long
memoryUsage;
};
string readFileValue(const string &path){
    ifstream file(path);  string value; if
(file.is_open()){
        getline(file,value);
    }
    return value;
}
double getSystemUptime(){
    ifstream file("/proc/uptime");
    double uptime;
```

```

        if (file.is_open()){
            file >> uptime;
        }

        return uptime;
    }

ProcessInfo getProcessInfo(int pid,double systemUptime){
    ProcessInfo pinfo;

    pinfo.pid = pid;    ifstream file("/proc/" +
to_string(pid) + "/stat");

        string line;    long utime=0,
stime=0, starttime=0;

    if(file.is_open()){
        getline(file,line);        istringstream
ss(line);        string token;

        int count=0;

        while(ss >> token){
            count++;

            if(count==2) pinfo.name=token;

            else if(count==14) utime=stol(token);

            else if(count==15) stime=stol(token);        else

            if(count==22) starttime=stol(token);

        }

    }

    ifstream memFile("/proc/" + to_string(pid) + "/status");

    if (memFile.is_open()){        string key, value,unit;

        while(memFile >> key >> value >> unit){

            if (key=="VmRSS"){

                pinfo.memoryUsage=stol(value);

                break;

            }

        }

    }

```

```

    }

    long total_time = utime + stime;    double seconds = systemUptime -
(starttime / sysconf(_SC_CLK_TCK));    pinfo.cpuUsage = (total_time /
sysconf(_SC_CLK_TCK)) / seconds * 100;    return pinfo;
}

vector<ProcessInfo> getAllProcesses(){
vector<ProcessInfo> processes;        double
systemUptime= getSystemUptime();

    for (const auto &entry : fs::directory_iterator("/proc")){
if (entry.is_directory()){            string
filename=entry.path().filename().string();            if
(all_of(filename.begin(),filename.end(), ::isdigit)){
                int pid=stoi(filename);

processes.push_back(getProcessInfo(pid,systemUptime));
            }
        }
    }

    return processes;
}

void sortProcesses(vector<ProcessInfo> &processes, bool sortByCpu){
    if(sortByCpu){
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo
&b)
            {
                return a.cpuUsage > b.cpuUsage;
            });
    }
    else{
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const
ProcessInfo &b)
            {
                return a.memoryUsage > b.memoryUsage;
            });
    }
}

```



```

    });

}

}

int main(){

    vector <ProcessInfo> processes = getAllProcesses();

    sortProcesses(processes,true);

    cout<<"PID\tCPU%\tMemory (kB)\tName\n";    for (size_t

i=0;i<min(processes.size(),size_t(10));i++){

    cout<<processes[i].pid<<"\t"<<processes[i].cpuUsage<<"%\t"<<processes[i].memory

Usage<<"%\t"<<processes[i].name<<"%\n";

    }

    return 0;

}

```

Day 3 Output

```

student@D001-38:~/Documents/capstone-3$ g++ --std=c++17 task3.cpp -o task3
student@D001-38:~/Documents/capstone-3$ ./task3
PID      CPU%      Memory (kB)      Name
3337     14.162%   140731003253864% (chrome)%
1615     2.83279%   140731003253864% (gnome-shell)%
2118     1.78108%   140731003253864% (chrome)%
2077     0.937703%  140731003253864% (chrome)%
1050     0.91241%   140731003253864% (mysqld)%
1486     0.875344%  140731003253864% (Xorg)%
4148     0.626913%  140731003253864% (nautilus)%
2390     0.621781%  140731003253864% (chrome)%
4076     0.337124%  140731003253864% (gedit)%
3795     0.225338%  140731003253864% (kworker/10:0-events)%
student@D001-38:~/Documents/capstone-3$ █

```

Day 4: Add functionality to kill processes.

```

#include <iostream>

#include <filesystem>

#include <fstream>

#include <sstream>

#include <algorithm>

#include <vector>

```

```

#include <dirent.h>

#include <unistd.h> #include
<csignal> using namespace
std; namespace
fs=std::filesystem; struct
ProcessInfo{
    int pid;    string
name;    double
cpuUsage;    long
memoryUsage;
};
string readFileValue(const string &path){
    ifstream file(path);    string value; if
(file.is_open()){
        getline(file,value);
    }
    return value;
}
double getSystemUptime(){
    ifstream file("/proc/uptime"); double
uptime;

    if (file.is_open()){
        file >> uptime;
    }
    return uptime;
}
ProcessInfo getProcessInfo(int pid,double systemUptime){
    ProcessInfo pinfo;

```

```

    pinfo.pid = pid;    ifstream file("/proc/" +
to_string(pid) + "/stat");

    string line;    long utime=0,
stime=0, starttime=0;

    if(file.is_open()){
        getline(file,line);        istringstream
ss(line);        string token;

        int count=0;

        while(ss >> token){
            count++;

            if(count==2) pinfo.name=token;

            else if(count==14) utime=stol(token);
else if(count==15) stime=stol(token);        else
if(count==22) starttime=stol(token);

        }

    }

    ifstream memFile("/proc/" + to_string(pid) + "/status"); if (memFile.is_open()){
        string key, value,unit;        while(memFile >> key >> value >> unit){

            if (key=="VmRSS"){

                pinfo.memoryUsage=stol(value);

                break;

            }

        }

    }

    long total_time = utime + stime;    double seconds = systemUptime -
(starttime / sysconf(_SC_CLK_TCK));    pinfo.cpuUsage = (total_time /
sysconf(_SC_CLK_TCK)) / seconds * 100;    return pinfo;

```

```
}
```

```
vector <ProcessInfo> getAllProcesses(){  
    vector <ProcessInfo> processes;          double  
    systemUptime= getSystemUptime();  
        for (const auto &entry : fs::directory_iterator("/proc")){  
if (entry.is_directory()){                string  
    filename=entry.path().filename().string();                if  
    (all_of(filename.begin(),filename.end(), ::isdigit)){  
                int pid=stoi(filename);  
  
    processes.push_back(getProcessInfo(pid,systemUptime));  
                }  
        }  
    }  
    return processes;  
}
```

```
void sortProcesses(vector<ProcessInfo> &processes, bool sortByCpu){  
    if(sortByCpu){  
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo  
&b)  
            {  
                return a.cpuUsage > b.cpuUsage;  
            });  
    }  
    else{  
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo  
&b)
```

```

        {
            return a.memoryUsage > b.memoryUsage;
        }
    });
}
}

```

```

int main(){
    vector <ProcessInfo> processes = getAllProcesses();
    sortProcesses(processes,true);
    cout<<"PID\tCPU%\tMemory (kB)\tName\n";    for (size_t
    i=0;i<min(processes.size(),size_t(10));i++){
    cout<<processes[i].pid<<"\t"<<processes[i].cpuUsage<<"%\t
    "<<processes[i].memory
    Usage<<"%\t"<<processes[i].name<<"%\n";

        }

    int targetPid;    cout<<"Enter PID to kill: ";    cin>>targetPid;
    if(targetPid > 0){    if (kill(targetPid, SIGKILL) == 0){
    cout<<"Process " << targetPid << " terminated successfully\n";

        }

        else{

            perror("Failed to kill the process");

        }

    }

    return 0;
}

```

Day 4 Output

```
student@D001-38:~/Documents/capstone-3$ g++ --std=c++17 task4.cpp -o task4
student@D001-38:~/Documents/capstone-3$ ./task4
PID      CPU%    Memory (kB)    Name
1615     3.07015%      140727110998360%    (gnome-shell)%
5939     2.64061%      140727110998360%    (chrome)%
1486     0.989632%     140727110998360%    (Xorg)%
4148     0.958026%     140727110998360%    (nautilus)%
1050     0.914171%     140727110998360%    (mysqld)%
5721     0.381869%     140727110998360%    (kworker/10:1-events)%
4076     0.351272%     140727110998360%    (gedit)%
3795     0.227279%     140727110998360%    (kworker/10:0-events)%
4170     0.123096%     140727110998360%    (gnome-terminal-)%
76       0.0478193%    140727110998360%    (ksoftirqd/10)%
Enter PID to kill: 5939
Process 5939 terminated successfully
student@D001-38:~/Documents/capstone-3$ █
```

Day 5: Implement real-time update feature to refresh data every few seconds.

```
#include <iostream>

#include <filesystem>

#include <fstream>

#include <sstream>

#include <algorithm>

#include <vector>

#include <dirent.h>

#include <unistd.h>

#include <csignal>

#include <thread> #include
<chrono> using namespace
std; namespace
fs=std::filesystem; struct
ProcessInfo{
    int pid;      string
    name;         double
    cpuUsage;     long
    memoryUsage;
```

```

};

string readFileValue(const string &path){
    ifstream file(path);  string value;  if
    (file.is_open()){      getline(file,value);

        }

        return value;
    }

double getSystemUptime(){
    ifstream file("/proc/uptime");

        double uptime;

        if (file.is_open()){

            file >> uptime;

        }

        return uptime;
    }

ProcessInfo getProcessInfo(int pid,double systemUptime){

    ProcessInfo pinfo;

        pinfo.pid = pid;  ifstream file("/proc/" +
    to_string(pid) + "/stat");

        string line;  long utime=0,
    stime=0, starttime=0;

    if(file.is_open()){

        getline(file,line);          istringstream
    ss(line);          string token;

    int count=0;

```

```

        while(ss >> token){
            count++;
            if(count==2) pinfo.name=token;

else if(count==14) utime=stol(token);           else
if(count==15) stime=stol(token);           else
if(count==22) starttime=stol(token);

        }
    }

    ifstream memFile("/proc/" + to_string(pid) + "/status");
    if (memFile.is_open()){        string key, value,unit;

        while(memFile >> key >> value >> unit){
            if (key=="VmRSS"){
                pinfo.memoryUsage=stol(value);

                break;
            }
        }
    }

    long total_time = utime + stime;    double seconds = systemUptime -
(starttime / sysconf(_SC_CLK_TCK));    pinfo.cpuUsage = (total_time /
sysconf(_SC_CLK_TCK)) / seconds * 100;    return pinfo;

}
vector <ProcessInfo> getAllProcesses(){

vector <ProcessInfo> processes;        double

systemUptime= getSystemUptime();

```



```

        for (const auto &entry : fs::directory_iterator("/proc")){
if (entry.is_directory()){
            string
filename=entry.path().filename().string();
            if
(all_of(filename.begin(),filename.end(), ::isdigit)){
                int pid=stoi(filename);

processes.push_back(getProcessInfo(pid,systemUptime));

            }

        }

    }

    return processes;
}

void sortProcesses(vector<ProcessInfo> &processes, bool sortByCpu){
    if(sortByCpu){
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo
&b)

            {

                return a.cpuUsage > b.cpuUsage;

            });

    }

    else{
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo
&b)

            {

                return a.memoryUsage > b.memoryUsage;

            });

    }
}

```

```
}
```

```
int main(){
```

```
    char input;    while
```

```
(true){
```

```
    system("clear");
```

```
        vector <ProcessInfo> processes =
```

```
    getAllProcesses();        sortProcesses(processes,true);
```

```
    cout<<"Press 'q' to quit";        cout<<"PID\tCPU%\tMemory
```

```
(kB)\tName\n";
```

```
        for (size_t i=0;i<min(processes.size(),size_t(10));i++){
```

```
    cout<<processes[i].pid<<"\t"<<processes[i].cpuUsage<<"%\t"<<processes[i].memory  
    Usage<<"%\t"<<processes[i].name<<"%\n";
```

```
        }
```

```
        int targetPid;
```

```
        cout<<"Enter PID to kill:
```

```
";        cin>>targetPid;
```

```
if (targetPid == 0){        //continue
```

```
refresh
```

```
        }
```

```
        else if(targetPid > 0){
```

```
if (kill(targetPid, SIGKILL) == 0){
```

```
        cout<<"Process "<< targetPid << " terminated  
successfully\n";
```

```
        }
```

```

        else{

            perror("Failed to kill the process");

        }

    }

    cout << "\nPress 'q' to quit or Enter to refresh: ";

    cin.ignore( );

    input = getchar( );

    if (input == 'q' || input == 'Q')

        break;

    this_thread::sleep_for(chrono::seconds(2));

}

return 0;

}

```

Day 5 Output (Killing Chrome Process)

```

Press 'q' to quitPID    CPU%    Memory (kB)    Name
6696    3.79939%    140737058960760%    (chrome)%
1615    3.13593%    140737058960760%    (gnome-shell)%
1486    1.02253%    140737058960760%    (Xorg)%
4148    1.01235%    140737058960760%    (nautilus)%
1050    0.911457%    140737058960760%    (mysqld)%
6586    0.845166%    140737058960760%    (gnome-terminal-)%
4076    0.331241%    140737058960760%    (gedit)%
3795    0.21504%    140737058960760%    (kworker/10:0-events)%
5721    0.186109%    140737058960760%    (kworker/10:1-events)%
76      0.0471295%    140737058960760%    (ksoftirqd/10)%
Enter PID to kill: 6696
Process 6696 terminated successfully

Press 'q' to quit or Enter to refresh: █

```

Day 5 Output (After Refreshing Process List)

```
Press 'q' to quitPID    CPU%    Memory (kB)    Name
1615    3.17135%    140737058960416%    (gnome-shell)%
4148    1.12394%    140737058960416%    (nautilus)%
1486    1.04837%    140737058960416%    (Xorg)%
1050    0.911807%    140737058960416%    (mysqld)%
6586    0.453206%    140737058960416%    (gnome-terminal-)%
4076    0.344642%    140737058960416%    (gedit)%
3795    0.210823%    140737058960416%    (kworker/10:0-events)%
5721    0.156335%    140737058960416%    (kworker/10:1-events)%
76      0.052087%    140737058960416%    (ksoftirqd/10)%
5182    0.0364611%    140737058960416%    (kworker/u24:1-events_unbound)%
Enter PID to kill: █
```